

作业 3

——Fingerprint——

1. 题目

描述

本题将会给你三个 $n \times n$ 的矩阵 A, B, C ，你需要判断 AB 是否等于 C 。

由于输入规模较大，我们在输入中只给出 C ，而 A, B 中的元素由以下函数生成：

```
int randint(const unsigned int &seed) {  
    static unsigned int x = seed;  
    x = (x \* 23333LL + 23327) % 1000000007;  
    return x % 20 - 10;  
}
```

其中 $seed$ 将会在输入中给出。生成矩阵 A, B 的代码如下：

```
for(int i = 1; i <= n; i ++)  
    for(int j = 1; j <= n; j++) {  
        a[i][j] = randint(seed);  
        b[i][j] = randint(seed);  
    }
```

输入

测试数据第一行为一个正整数 $n \leq 3000$ ，表示矩阵的大小。第二行为一个不超过 10^9 的正整数，表示 $seed$ 。接下来 n 行，每行包含用空格分开的 n 个整数，表示矩阵 C 。

输出

输出 “YES” 或 “NO”（不包含引号），表示 $AB = C$ 是否成立。

样例输入

```
3  
462191211  
2 -67 106  
102 42 -2  
-63 -57 47
```

样例输出

NO

2. 思路

由于矩阵的大小 n 可以达到 3000，直接计算 AB 的时间复杂度为 $O(n^3)$ ，这在实际中是不可行的。我们可以使用随机化算法来验证 AB 是否等于 C 。

由数学知识，我们知道如果 $AB = C$ ，那么对于任意向量 x ，都有 $A(Bx) = Cx$ 。因此，我们可以随机生成一个向量 x ，计算 Bx 和 Cx ，然后再计算 $A(Bx)$ ，最后比较 $A(Bx)$ 和 Cx 是否相等。

在这个算法中，我们能够会得到：

- 1) 如果 $AB = C$ ，我们一定会输出 “YES”。
- 2) 如果 $AB \neq C$ ，我们可能输出 “YES”，只是这个概率应该足够低。

我们需要的数学知识：

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 + \dots + a_{2n} \cdot x_n \\ \vdots \\ a_{n1} \cdot x_1 + a_{n2} \cdot x_2 + \dots + a_{nn} \cdot x_n \end{pmatrix}$$

因此，我们可以创建一个函数来进行 $n \times n$ 矩阵与 n 维向量乘法，这样可以简化主函数的代码。

2.1. 解题步骤

首先，我们需要按照要求生成矩阵 A, B ，然后输入 C 矩阵。

接着，我们来书写矩阵与向量的乘法函数：我们需要传入一个 $n \times n$ 的矩阵和一个 n 维的向量，输出一个 n 维的结果向量。通过之前的数学知识铺垫，我们容易通过循环求出这个向量的各个分量。

```
void MatrixMultiplyVector(const vector<vector<int>> &A, const vector<int> &x, vector<int> &res) {
    int n = A.size();
    for(int i = 0; i < n; i++) {
        res[i] = 0;
        for(int j = 0; j < n; j++) {
            res[i] += A[i][j] * x[j];
        }
    }
}
```

之后，我们需要生成一些随机地 n 维列向量，来进行检测。为了避免一些可能的巧合，我们使用我们自己的随机函数来生成这个向量。（我不确定用题目给的这个随机函数效果如何，也许也可以实现目标。）

最后我们只需要检查 $A(Bx)$ 和 Cx 是否相等，如果不相等，我们就输出“NO”，如果相等，我们继续进行下一轮的检测，直到完成所有的检测之后才输出“YES”。

还是老规矩，我们只有数学推导才能严格说明应该生成检测几个随机的 x ，因此在完成这个题目的时候，我们可以先设置一个常数 TIMES 来控制检测的次数，如果精度不够，我们只需要调整这个数据取值即可。

2.2. 代码实现

```
#include<bits/stdc++.h>
using namespace std;
const int TIMES = 10;
int randint(const unsigned int &seed) {
    static unsigned int x = seed;
    x = (x * 23333LL + 23327) % 1000000007;
    return x % 20 - 10;
}
void MatrixMultiplyVector(const vector<vector<int>> &A, const vector<int> &x, vector<int> &res) {
    int n = A.size();
    for(int i = 0; i < n; i++) {
        res[i] = 0;
        for(int j = 0; j < n; j++) {
            res[i] += A[i][j] * x[j];
        }
    }
}
int main(){
    int n;
    cin >> n;
    vector<vector<int>> a(n,vector<int>(n)),b(n,vector<int>(n)),c(n,vector<int>(n));
```

```

long long seed;
cin >> seed;
// Matrix C is input here
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        scanf("%d", &c[i][j]);    //scanf is faster than cin.

// Matrix A and B are generated here
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++) {
        a[i][j] = randint(seed);
        b[i][j] = randint(seed);
    }
}
srand(time(0));    // To avoid some possible coincidences, we use our own random function.

for(int i = 0; i < TIMES; i++){
    // randomly generate a vector in n dimensions
    vector<int> x(n);
    for(int i = 0; i < n; i++)
        x[i] = rand() % 100000;
    vector<int> ResBX(n), ResCX(n), ResABX(n);
    MatrixMultiplyVector(b, x, ResBX);
    MatrixMultiplyVector(c, x, ResCX);
    MatrixMultiplyVector(a, ResBX, ResABX);
    if(ResABX != ResCX) {
        cout << "NO" << endl;
        return 0;
    }
    else
        continue;
}
cout << "YES" << endl;
return 0;
}

```